

Number Patterns

Number patterns are a great way to practice and understand programming's nested loops and output formatting. These patterns, ranging from simple sequences to complex arrangements, help in enhancing logical thinking and problem-solving skills in Python.

1. Pattern 1

```
1
12
123
1234
12345
```

Python Code:

```
n = 5
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print(j, end=' ')
    print()
```

Explanation:

- The outer loop runs from 1 to n.
- The inner loop prints numbers from 1 to the current value of the outer loop (i).

2. Pattern 2

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

Python Code:

```
rows = 6
# if you want user to enter a number, uncomment the below line
# rows = int(input('Enter the number of rows'))
# outer loop
for i in range(rows):
    # nested loop
    for j in range(i):
        # display number
        print(i, end=' ')
    # new line after each row
    print("")
```

3. Pattern 3 Inverted Pyramid Pattern of Numbers

```
1 1 1 1 1
2 2 2 2
3 3 3
4 4
5
```

Python Code:

```
rows = 5
b = 0
# reverse for loop from 5 to 0
for i in range(rows, 0, -1):
    b += 1
    for j in range(1, i + 1):
        print(b, end=' ')
    print('\r')
```

Assignment:

Create a below-number pattern for loops.

1. Inverted Pyramid pattern with the same digit

```
5 5 5 5 5
5 5 5 5
5 5 5
5 5
5
```

2. Inverted half-pyramid pattern with a number

```
0 1 2 3 4 5
0 1 2 3 4
0 1 2 3
0 1 2
0 1
```

3. Reverse Number Pattern.

```
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

4. Reverse Pyramid Pattern.

```
1
2 1
3 2 1
4 3 2 1
5 4 3 2 1
```

5. Reverse Pattern.

```
5 4 3 2 1
4 3 2 1
3 2 1
2 1
1
```

6. Print Reverse Numbers 1 to 10.

```
1
3 2
6 5 4
10 9 8 7
```

7. Number Triangle Pattern.

```
    1
   1 2
  1 2 3
 1 2 3 4
1 2 3 4 5
```

8. Square Pattern Numbers.

```
1 2 3 4 5
2 2 3 4 5
3 3 3 4 5
4 4 4 4 5
5 5 5 5 5
```

9. Multiplication Table Pattern.

```
1
2 4
3 6 9
4 8 12 16
5 10 15 20 25
6 12 18 24 30 36
7 14 21 28 35 42 49
8 16 24 32 40 48 56 64
```

10. Pascal's Triangle Pattern using numbers.

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
```